Gangli's Algorithm 4 (MMJ distance by Calculation and Copy)

Worked for 29s >

Gangli's Algorithm 4 (MMJ distance by Calculation and Copy) is an $O(n^2)$ algorithm by Gangli Liu for computing the all-pairs minimax path distances (also called Min-Max-Jump / MMJ distance, longest-leg path distance, or all-points path distance / APPD matrix) in an undirected dense graph. arXiv

It is the first practical, coded implementation that achieves true $O(n^2)$ time for this problem on undirected dense graphs (where a complete graph on n points has $\Theta(n^2)$ edges). Theoretical $O(n^2)$ claims existed earlier, but lacked working code implementations.

Background

The **minimax path** (or MMJ distance) between two nodes i and j is the path that *minimizes the maximum edge weight* along the path:

$$MMJ(i, j \mid G) = \min_{\text{paths } p: i \to j} \max_{e \in p} w(e)$$

This is equivalent (by sign-flipping weights) to the classic widest-path / maximum-capacity path problem.

A key fact used by the algorithm: in an undirected graph, the unique path between any two nodes in a **Minimum Spanning Tree (MST)** is a minimax path.

The Algorithm

text Copy

```
Algorithm 4: MMJ distance by Calculation and Copy
Input:  Set of n points Ω (or equivalently the complete undirected graph)
Output: n × n MMJ / APPD matrix M

1. Initialize M with zeros
2. Construct an MST T of the complete graph on Ω          // O(n²) with Prim
3. Sort the edges of T in descending order of weight → list L
4. for each edge e in L (largest to smallest):
      Remove e from T → produces two connected components (sub-trees) T₁ and T₂
      for every node p ∈ T₁ and every node q ∈ T₂:
          M[p,q] ← M[q,p] ← weight(e)
5. return M
```

Why it is $O(n^2)$ (despite the nested loops looking cubic):

Each of the $\binom{n}{2}$ matrix entries is written *exactly once*. When an edge is removed early (large weight), the two sub-trees are large; when removed late (small weight), they are tiny. The total work across all removals is therefore $\Theta(n^2)$.

An equivalent formulation exists that sorts edges *ascending* and *adds* them one-by-one to an empty forest (union-find style).

Implementation & Performance

- Source code (Python + C++ versions, parallel variants, notebooks): https://github.com/mike-liuliu/Algorithm_4
- Paper describing the efficient implementation and extensive timing experiments: arXiv:2407.07058 ("An efficient implementation for solving the all pairs minimax path problem in an undirected dense graph").

On a typical desktop, Algorithm 4 computes the full APPD matrix for 10 000 points in roughly a minute, while classic $O(n^3)$ methods (modified Floyd-Warshall, recursive Algorithm 1, MST + all-pairs shortest paths) cannot finish within two hours.

Context in Liu's work

It is one of a family of algorithms named "Gangli's Algorithm X":

- Algorithm 1 – MMJ by recursion ($O(n^3)$)
- Algorithm 4 – MMJ by Calculation and Copy ($O(n^2)$, this one)
- later algorithms add warm-start, parallel acceleration, etc.

MMJ distance itself is useful for clustering non-convex / elongated clusters, improved internal validation indices (MMJ-Silhouette, MMJ-CH, MMJ-DB), and label prediction of new points after clustering.

30 sources